CMPT 310 - Project Milestone 2

Project Recap

Our project is an AI agent that learns to play the game 2048 autonomously. The input is the current 4×4 game board, and the output is one of four possible actions (up, down, left, or right). We use reinforcement learning as our primary AI approach, with a Random agent and our previous Expectimax agent as baseline methods for comparison.

Accomplishments with Proof

Accomplishment 1: Implemented N-Tuple Network Value Function

To enable reinforcement learning, we replaced the manually designed heuristic evaluation function from our Expectimax agent with a learned value function. We implemented an N-tuple network that estimates the value of a given 2048 board state by learning patterns from board configurations. The network represents the 4×4 game board as multiple groups of tuples. Each tuple maps the observed tile values into lookup table entries, and the overall value of a board is calculated by summing the values from all tuple patterns. To improve generalization, we implemented symmetry handling by evaluating rotated and reflected versions of the board. The N-tuple network serves as the value function $V(s)$, where higher values represent board states that are expected to achieve higher future rewards. Initially, all lookup table values are initialized to zero, and they are gradually updated during training through experience generated from self-play.

Proof: code excerpts from `agents/n_tuple_network.py`

```python
Coordinate = Tuple[int, int]
TuplePattern = Tuple[Coordinate, ...]
PathLike = Union[str, "Path"]


def _default_tuples() -> Tuple[TuplePattern, ...]:
    rows = [tuple((row, col) for col in range(BOARD_SIZE)) for row in range(BOARD_SIZE)]
    columns = [tuple((row, col) for row in range(BOARD_SIZE)) for col in range(BOARD_SIZE)]
    squares = [
        tuple((row + dr, col + dc) for dr in range(2) for dc in range(2))
        for row in range(BOARD_SIZE - 1)
        for col in range(BOARD_SIZE - 1)
    ]
    return tuple(rows + columns + squares)


DEFAULT_TUPLES = _default_tuples()
```

```python
    def feature_indices(self, grid: np.ndarray, validate: bool = True) -> np.ndarray:
        """Return the table index selected by each symmetry and tuple.

        Pass validate=False in hot loops once the caller guarantees a legal board.
        """
        board = np.asarray(grid)
        if board.shape != (BOARD_SIZE, BOARD_SIZE):
            raise ValueError("grid must have shape (4, 4)")

        ranks = self._ranks_checked(board) if validate else _RANK_LUT[board]

        selected = ranks.reshape(-1)[self._symmetry_cells].astype(np.int64, copy=False)
        return selected @ self._place_values

    def value(self, grid: np.ndarray, validate: bool = True) -> float:
        indices = self.feature_indices(grid, validate=validate)
        table_numbers = np.arange(len(self.tuples), dtype=np.intp)[None, :]
        return float(self.tables[table_numbers, indices].sum(dtype=np.float64))
```

```python

    def update(self, grid: np.ndarray, delta: float) -> None:
        """Distribute delta evenly over every table lookup used by value()."""
        try:
            numeric_delta = float(delta)
        except (TypeError, ValueError) as error:
            raise TypeError("delta must be a finite real number") from error
        if not np.isfinite(numeric_delta):
            raise ValueError("delta must be finite")

        indices = self.feature_indices(grid, validate=False)
        amount_per_lookup = numeric_delta / indices.size
        offsets = np.arange(len(self.tuples), dtype=np.int64) * TABLE_SIZE
        np.add.at(self.tables.reshape(-1), (indices + offsets).reshape(-1), amount_per_lookup)
```

The excerpts show the 17 tuple patterns extracted from the board (4 rows, 4 columns, 9 overlapping 2×2 squares), the conversion of a board into lookup table indices across all 8 symmetries, and the paired `value()` and `update()` methods.

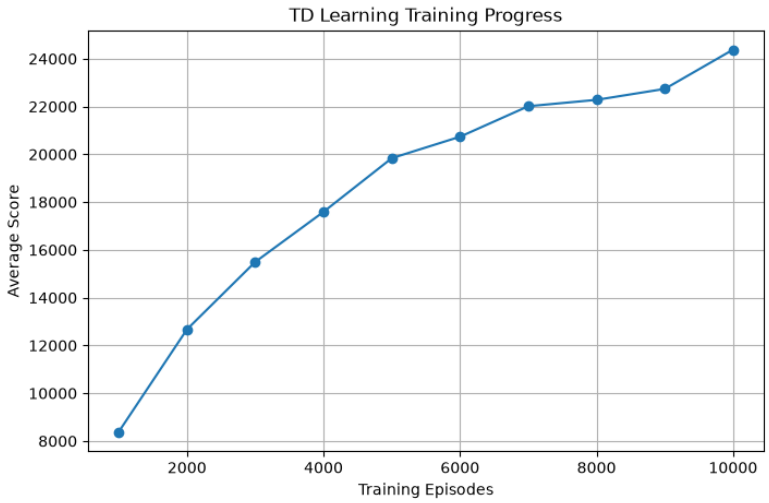
This replaces the hand-designed evaluation function from Milestone 1 with one that is learned. The symmetry handling is what makes the learning practical: the eight rotations and reflections of a board share the same lookup tables, so one observed position writes into $8 \times 17 = 136$ table entries at once and the agent generalizes from far fewer games. The `value()` and `update()` pair is the interface TD learning needs, which is what let the training pipeline in Accomplishment 2 be built on top of it.

Accomplishment 2: Implemented TD Learning Self-Play Training Pipeline

We implemented a Temporal Difference (TD) learning training pipeline that allows the agent to improve its decision-making through self-play. The agent generates its own experience by repeatedly playing complete 2048 games. During each episode, the agent observes the current board state, selects the move with the highest estimated value using the current N-tuple network, performs the action, and receives a reward based on the score gained from tile merges. The TD error is calculated by comparing the current value estimate with the observed reward and the estimated future value. Then, the N-tuple

network is updated using this error to gradually improve its prediction. We also implemented checkpoint saving that saves learned values periodically (every 1000 episodes). This helps us track progress and enables future evaluation or continued training.

Proof: training progress graph and the TD update in `training/td_learning.py`




```
        best_action = None
        best_afterstate = None
        best_reward = 0.0
        best_value = float("-inf")

        for action, move in MOVES.items():
            afterstate, merged_values, changed = move(state)
            if not changed:
                continue

            reward = points_after_merge(merged_values)
            value = reward + self.network.value(afterstate, validate=False)

            if value > best_value:
                best_action = action
                best_afterstate = afterstate
                best_reward = reward
                best_value = value

        return best_action, best_afterstate, best_reward, best_value

    def play_episode(self):
        """One self-play game with TD(0) updates over afterstates.

        V(afterstate) estimates FUTURE reward only, so the target for afterstate A1 is
        R2 + V(A2), where R2 is the reward of the move taken from the NEXT board.
        The current move's own reward is used for action selection, not for the target.
        """
```

The console output shows the average score per 1000 episodes rising as self-play proceeds, and the graph plots that same progression over $10{,}000$ episodes, from roughly $8{,}400$ to $24{,}400$.

The value function starts at zero everywhere, so all of this improvement comes from experience the agent generated itself. The curve rising steadily and not collapsing is evidence the TD updates are stable, which is the main risk with a learning rate applied to a lookup table.

Accomplishment 3: Integrated RL Agent with Existing 2048 System and Completed Interactive UI

We integrated the reinforcement learning components with our existing 2048 environment, which includes board movement, tile merging, random tile spawning, scoring, and game termination logic. The reinforcement learning pipeline was designed to interact directly with this environment, allowing the N-tuple network value function to learn board evaluations through self-play without modifying the core game mechanics. Additionally, we completed the graphical user interface using Pygame. It includes a main menu where users can select between the agents, a 4×4 game board, score and best score display, and tile animations for movement and merging.

Proof: the evaluation function that lets one search use either value function, and a table comparing the previous Expectimax agent, the Expectimax agent using the learned value function, and the Random baseline

```
    #  Learned value function or the previous heuristic can be used.
    def evaluate(self, state):
        if self.network is not None:
            return self.network.value(state, validate=False)
        empty = len(empty_cells(state))
        max_tile = np.max(state)
        return (100 * empty) + max_tile
```

Agent	Games	Avg Score	Median	Best	Best Tile	Reached 2048
Random	15	992	720	2,772	256	0/15
Expectimax (heuristic)	15	9,974	7,184	26,768	2048	1/15
Expectimax + RL	15	48,487	37,880	78,224	4096	15/15

All three agents played 15 games on the same environment, with the RL agent using the trained value function inside the same Expectimax search at depth 2.

The learned value function outperforms the hand-written heuristic by roughly $4.9\times$ and reaches the 2048 tile in every game, where the heuristic reaches it once in 15 and the random baseline never does. The comparison also isolates what the search contributes: the training graph shows the value function alone averaging about $24{,}400$ when picking moves greedily, and placing that same network inside the depth-2 search roughly doubles

it. Both components are needed, which is why the heuristic Expectimax is kept as a baseline rather than replaced.

Proof for UI: video showing everything from the main menu to some moves being played, to show tile animations and score updates.

Challenges

The main challenge we encountered during Milestone 2 was determining how to design and implement the reinforcement learning component of our agent. The RL agent required decisions about the value representation, training algorithm, and learning process. We needed to carefully consider how the board state should be represented and how the agent could learn useful patterns from experience.

We researched different possible approaches for representing the value function, including neural network-based methods, however, we realized that a neural network would introduce additional complexity in terms of architecture design and computational requirements. We decided to use an N-tuple network combined with Temporal Difference learning. This approach is computationally efficient, well-suited for the 2048 environment, and allows the agent to learn board patterns through self-play using lookup table updates.

Changes from Original Plan

There are no changes to our original project scope or AI approach. We continued with our original goal of developing an AI agent capable of autonomously playing 2048 and evaluating its performance against baseline approaches. The main adjustments made during development were implementation decisions rather than changes to the project direction. For Milestone 2, we implemented reinforcement learning using a Temporal Difference (TD) learning algorithm with an N-tuple network as the value representation. This approach was selected because it is well-suited for the 2048 environment, computationally efficient for self-play training, and allows the agent to learn board patterns through experience. These design choices allowed us to continue following our original plan while keeping the project scope achievable within the given timeline.

GitHub repo link: `https://github.com/praneetkb/2048-Master`